Web Banking Application – Execution Plan

This document presents a clear, structured execution plan for a Web Banking Application. It translates functional requirements into concrete architecture, APIs, data models, and phased deliverables suitable for real-world implementation.

1. System Architecture (High Level)

Frontend:

- React + TypeScript
- Material UI
- JWT stored in HttpOnly cookie or memory
- Protected routes

Backend:

- Node.js + Express (TypeScript)
- JWT authentication
- REST API (Swagger documented)
- WebSocket (Socket.IO)
- Jitsi integration
- OpenAI SDK (chatbot)

Database:

- MongoDB
- Mongoose ODM

Infrastructure:

- Docker (Backend)
- Hosting: Render / Railway / Vercel / AWS

2. Core Functional Flow

Sign-Up Flow:

- User enters email, password, and phone number
- Backend validates input and email uniqueness
- Generate 6-digit OTP
- Send OTP via SMS (Twilio) or Email

- User submits OTP
- Backend validates OTP and activates user
- Random balance assigned
- JWT issued and user redirected to Dashboard

Login Flow:

- User submits email and password
- Password verified using bcrypt
- JWT issued on success
- Redirect to Dashboard

Dashboard:

- Protected route
- Displays account balance and recent transactions
- Logout clears JWT

Money Transfer Flow:

- Sender inputs receiver email and amount
- Backend validates receiver and balance
- Atomic balance update for sender and receiver
- Create transaction records for both sides
- Notify receiver via WebSocket
- Optional Jitsi video call initiation

3. REST API (Core Endpoints)

- POST /api/auth/register
- POST /api/auth/verify-otp
- POST /api/auth/login
- POST /api/auth/logout
- GET /api/user/me
- POST /api/transactions/transfer
- GET /api/transactions/history
- POST /api/support/chat

4. MongoDB Schemas

User Schema:

```
{
  email: string (unique),
```

```
  passwordHash: string,
  phone: string,
  isVerified: boolean,
  balance: number,
  createdAt: Date
}
```

Transaction Schema:

```
{
  from: ObjectId | null,
  to: ObjectId | null,
  amount: number,
  type: "SEND" | "RECEIVE",
  createdAt: Date
}
```

OTP Schema:

```
{
  userId: ObjectId,
  code: string,
  expiresAt: Date
}
```

5. Security Considerations

- Password hashing using bcrypt
- Short-lived JWT tokens
- OTP expiration (5–10 minutes)
- Rate limiting for login and OTP
- Strict input validation (Joi / Zod)
- MongoDB transactions for money transfers

6. Phase-by-Phase Execution Plan

- Phase 1 – Design (2 Days): Swagger definitions, sequence diagrams, JWT flow
- Phase 2 – Backend Core (2 Days): Express server, auth & transfer logic, Postman tests
- Phase 3 – Database (1 Day): MongoDB schemas and Mongoose models
- Phase 4 – Docker (1 Day): Dockerfile and docker-compose
- Phase 5 – UI/UX (0.5 Day): Figma designs

- Phase 6 – Frontend (2 Days): Routing, auth context, API integration
- Phase 7 – WebSocket (2 Days): Real-time notifications
- Phase 8 – Video Call (2 Days): Jitsi integration
- Phase 9 – Chatbot (1 Day): OpenAI-powered support bot
- Phase 10 – Backend Testing (1 Day): Jest and Supertest
- Phase 11 – Frontend Testing (1 Day): Cypress E2E tests